A Visual Pricing Engine for European Options under the Black-Scholes Model

Motivation

Option pricing lies at the core of modern financial risk management, trading, and insurance-linked investment strategies. Financial institutions rely on robust, transparent pricing models to value derivative contracts, assess hedging effectiveness, and understand the sensitivities of portfolios to market movements.

The Black-Scholes model remains one of the most influential frameworks in derivative pricing and is widely used as a benchmark in industry, despite its simplifying assumptions. This project replicates a professional workflow by combining mathematical theory, clean object-oriented implementation, and a visual dashboard to enhance interpretability and stakeholder communication.

By developing a visual pricing engine for European call and put options, this project mirrors the type of analytical tooling used by quantitative researchers, and risk teams when validating pricing assumptions, communicating model outputs, and performing sensitivity analysis.

Theory

This project is based on the Black-Scholes-Merton framework for pricing European-style options in an arbitrage-free market.

Definition (Geometric Brownian Motion)

The price process $\{S_t\}_{t\geq 0}$ of the underlying asset is assumed to follow a geometric Brownian motion of the form

$$\mathrm{d}S_t = \mu S_t \,\mathrm{d}t + \sigma S_t \,\mathrm{d}W_t,$$

where μ is the drift, σ is the volatility, and W_t denotes a standard Brownian motion.

Under this assumption, logarithmic returns are normally distributed and asset prices remain strictly positive.

No-Arbitrage and Risk-Neutral Valuation

Theorem (Risk-Neutral Measure)

In an arbitrage-free market, there exists an equivalent probability measure $\mathbb{Q}$ under which the discounted asset price process

$$\tilde{S}_t = e^{-rt} S_t$$

is a martingale.

Under the risk-neutral measure, the drift μ is replaced by the risk-free interest rate r, yielding

$$\mathrm{d}S_t = rS_t \,\mathrm{d}t + \sigma S_t \,\mathrm{d}W_t^{\mathbb{Q}}.$$

Black-Scholes Partial Differential Equation

Let $V(S,t)$ denote the price of a European derivative written on S_t. Applying Itô's Lemma and constructing a self-financing hedged portfolio leads to the Black-Scholes partial differential equation:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0.$$

For a European call option with maturity T and strike K, the terminal condition is

$$V(S,T) = \max(S - K, 0).$$

Closed-Form Solution

Solving the PDE subject to the terminal condition yields the well-known Black-Scholes formula for a European call option:

$$C(S,t) = S_t N(d_1) - Ke^{-r(T-t)}N(d_2),$$

where

$$d_1 = \frac{\ln\left(\frac{S_t}{K}\right) + \left(r + \frac{1}{2}\sigma^2\right)(T-t)}{\sigma\sqrt{T-t}}, \quad d_2 = d_1 - \sigma\sqrt{T-t}.$$

The corresponding European put price is given by

$$P(S,t) = Ke^{-r(T-t)}N(-d_2) - S_t N(-d_1).$$

Here, $N(\cdot)$ denotes the cumulative distribution function of the standard normal distribution.

Interpretation

The quantities $N(d_1)$ and $N(d_2)$ admit probabilistic interpretations under the risk-neutral measure and encode the sensitivity of the option value to the underlying asset price and time to maturity. Despite its simplifying assumptions, the Black-Scholes model provides a foundational benchmark for derivative pricing and risk management.

Data Requirements

- Underlying asset spot price (real-time or user-specified).
- Option contract specifications (strike price and maturity).
- Risk-free interest rate (annualised).
- Volatility assumption (historical estimate or implied input).
- No external datasets are required; inputs are provided interactively.

Code

The following code excerpts illustrate production-quality implementations in both Python and C++.

Python: Object-Oriented Pricing Engine

```python
import numpy as np
from scipy.stats import norm

class BlackScholesModel:
    """
    Implementation of the Black-Scholes pricing model
    for European call and put options.
    """

    def __init__(self, spot, strike, rate, maturity, volatility):
        self.spot = spot
        self.strike = strike
        self.rate = rate
        self.maturity = maturity
        self.volatility = volatility

    def _d1_d2(self):
        d1 = (
            np.log(self.spot / self.strike)
            + (self.rate + 0.5 * self.volatility**2) * self.maturity
        ) / (self.volatility * np.sqrt(self.maturity))
        d2 = d1 - self.volatility * np.sqrt(self.maturity)
        return d1, d2

    def call_price(self):
        d1, d2 = self._d1_d2()
        return (
            self.spot * norm.cdf(d1)
            - self.strike * np.exp(-self.rate * self.maturity) * norm.cdf(d2)
        )

    def put_price(self):
        d1, d2 = self._d1_d2()
        return (
            self.strike * np.exp(-self.rate * self.maturity) * norm.cdf(-d2)
            - self.spot * norm.cdf(-d1)
        )
```

This class is integrated into a Streamlit dashboard, enabling real-time visualisation of option prices as model parameters change.

Python: Streamlit Visual Pricing Dashboard

```python
import streamlit as st
import pandas as pd
from black_scholes_model import BlackScholesModel

st.set_page_config(
    page_title="Visual Black-Scholes Pricing Engine",
    layout="wide"
)

st.title("A Visual Pricing Engine for European Options")
```

```python

st.markdown(
    """
    This dashboard provides an interactive implementation of the
    Black-Scholes model for pricing European call and put options.
    Users can explore how option values respond to changes in
    volatility, interest rates, and time to maturity.
    """
)

st.markdown("---")

with st.sidebar:
    st.header("Model Parameters")

    spot = st.number_input(
        "Spot Price",
        min_value=0.01,
        value=100.0,
        step=1.0
    )

    strike = st.number_input(
        "Strike Price",
        min_value=0.01,
        value=100.0,
        step=1.0
    )

    maturity = st.number_input(
        "Time to Maturity (years)",
        min_value=0.01,
        value=1.0,
        step=0.25
    )

    volatility = st.slider(
        "Volatility",
        min_value=0.01,
        max_value=1.0,
        value=0.2
    )

    rate = st.slider(
        "Risk-Free Interest Rate",
        min_value=0.0,
        max_value=0.2,
        value=0.05
    )

model = BlackScholesModel(
    spot=spot,
    strike=strike,
    rate=rate,
    maturity=maturity,
    volatility=volatility
)

```

```
call_price = model.call_price()
put_price = model.put_price()

st.subheader("Option Valuation")

col1, col2 = st.columns(2)

with col1:
    st.metric(
        label="Call Option Value",
        value=f"{call_price:.2f}"
    )

with col2:
    st.metric(
        label="Put Option Value",
        value=f"{put_price:.2f}"
    )

st.markdown("---")

st.subheader("Model Inputs")

parameters = pd.DataFrame({
    "Parameter": [
        "Spot Price",
        "Strike Price",
        "Time to Maturity",
        "Volatility",
        "Risk-Free Rate"
    ],
    "Value": [
        spot,
        strike,
        maturity,
        volatility,
        rate
    ]
})

st.dataframe(parameters, hide_index=True)
```

C++: Simple Pricing Tool

```
#include <iostream>
#include <cmath>

double normalCDF(double x) {
    return 0.5 * std::erfc(-x / std::sqrt(2));
}

int main() {
    double S = 100.0;    // Spot price
    double K = 100.0;    // Strike price
    double r = 0.05;     // Risk-free rate
    double T = 1.0;      // Time to maturity
    double sigma = 0.2; // Volatility
```

```
14
15      double d1 = (std::log(S / K) + (r + 0.5 * sigma * sigma) * T)
16                  / (sigma * std::sqrt(T));
17      double d2 = d1 - sigma * std::sqrt(T);
18
19      double callPrice = S * normalCDF(d1)
20          - K * std::exp(-r * T) * normalCDF(d2);
21
22      double putPrice = K * std::exp(-r * T) * normalCDF(-d2)
23          - S * normalCDF(-d1);
24
25      std::cout << "Call Price: " << callPrice << std::endl;
26      std::cout << "Put Price:  " << putPrice << std::endl;
27
28      return 0;
29  }
```

Expected Outputs

- Theoretical European call and put prices.
- Interactive visual dashboard for parameter sensitivity analysis.
- Clear comparison of pricing behaviour across volatility and maturity.
- Improved intuition around option convexity and time value.

Interview Questions

1. What assumptions underpin the Black-Scholes model, and where do they fail in practice?
2. How would you extend this framework to include dividends or stochastic volatility?
3. How would you validate the pricing engine against observed market prices?

Resume Bullets

- Built an object-oriented Black-Scholes pricing engine with an interactive Streamlit dashboard.
- Applied derivative pricing theory to develop a transparent and explainable valuation tool.
- Implemented pricing logic in both Python and C++, demonstrating cross-language proficiency.